

FEATURE ENGINEERING

NBA Player Longevity Prediction

Project Overview

This notebook focuses on analyzing and preparing an NBA player performance dataset for machine learning. The goal is to transform raw sports statistics into model-ready inputs to forecast player longevity (`target_5yrs`).

Core Tasks:

1. Load dataset and define the target variable.
2. Drop non-predictive features to prevent data leakage.
3. Perform correlation analysis to reduce redundancy.
4. Engineer composite performance metrics.
5. Handle null values and clean data for ML readiness.

Step 1: Environment Setup and Data Loading

```
In [1]: ## Step 2: Environment Setup and Data Loading
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# Load the dataset.
df = pd.read_csv('nba_players.csv')

# Display basic info and first few rows
print(df.info())
print("\nTarget Variable Distribution:")
print(df['target_5yrs'].value_counts(dropna=False))

df.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1340 entries, 0 to 1339
Data columns (total 22 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Unnamed: 0      1340 non-null   int64
1   name            1340 non-null   object
2   gp              1340 non-null   int64
3   min             1340 non-null   float64
4   pts             1340 non-null   float64
5   fgm             1340 non-null   float64
6   fga             1340 non-null   float64
7   fg              1340 non-null   float64
8   3p_made         1340 non-null   float64
9   3pa             1340 non-null   float64
10  3p              1340 non-null   float64
11  ftm             1340 non-null   float64
12  fta             1340 non-null   float64
13  ft              1340 non-null   float64
14  oreb            1340 non-null   float64
15  dreb            1340 non-null   float64
16  reb             1340 non-null   float64
17  ast             1340 non-null   float64
18  stl             1340 non-null   float64
19  blk             1340 non-null   float64
20  tov             1340 non-null   float64
21  target_5yrs     1340 non-null   int64
dtypes: float64(18), int64(3), object(1)
memory usage: 230.4+ KB
None
```

Target Variable Distribution:

target_5yrs

1 831

0 509

Name: count, dtype: int64

Out[1]:

	Unnamed: 0	name	gp	min	pts	fgm	fga	fg	3p_made	3pa	...	fta	ft	ore
0	0	Brandon Ingram	36	27.4	7.4	2.6	7.6	34.7	0.5	2.1	...	2.3	69.9	0
1	1	Andrew Harrison	35	26.9	7.2	2.0	6.7	29.6	0.7	2.8	...	3.4	76.5	0
2	2	JaKarr Sampson	74	15.3	5.2	2.0	4.7	42.2	0.4	1.7	...	1.3	67.0	0
3	3	Malik Sealy	58	11.6	5.7	2.3	5.5	42.6	0.1	0.5	...	1.3	68.9	1
4	4	Matt Geiger	48	11.5	4.5	1.6	3.0	52.4	0.0	0.1	...	1.9	67.4	1

5 rows × 22 columns



Step 2: Remove Non-Predictive Features

Columns such as player names and internal database IDs add unique noise and present a risk of data leakage. They are removed to ensure the model focuses purely on mathematical performance statistics.

```
In [2]: # Identify columns to drop.
non_predictive_cols = ['player_name', 'id', 'player_id']

# Drop columns safely if they exist in the dataframe.
df_cleaned = df.drop(columns=[col for col in non_predictive_cols if col in df.columns])
features_only = df_cleaned.drop(columns=['target_5yrs'])

print(f"Remaining columns after drop: {df_cleaned.shape[1]}")
```

Remaining columns after drop: 22

Step 3: Correlation Analysis

columns that carry highly identical information (multicollinearity) to simplify your feature space.

```
In [3]: # Calculate correlation matrix on numeric features (excluding target variable)
features_only = df_cleaned.drop(columns=['target_5yrs'])
corr_matrix = features_only.corr(numeric_only=True).abs()

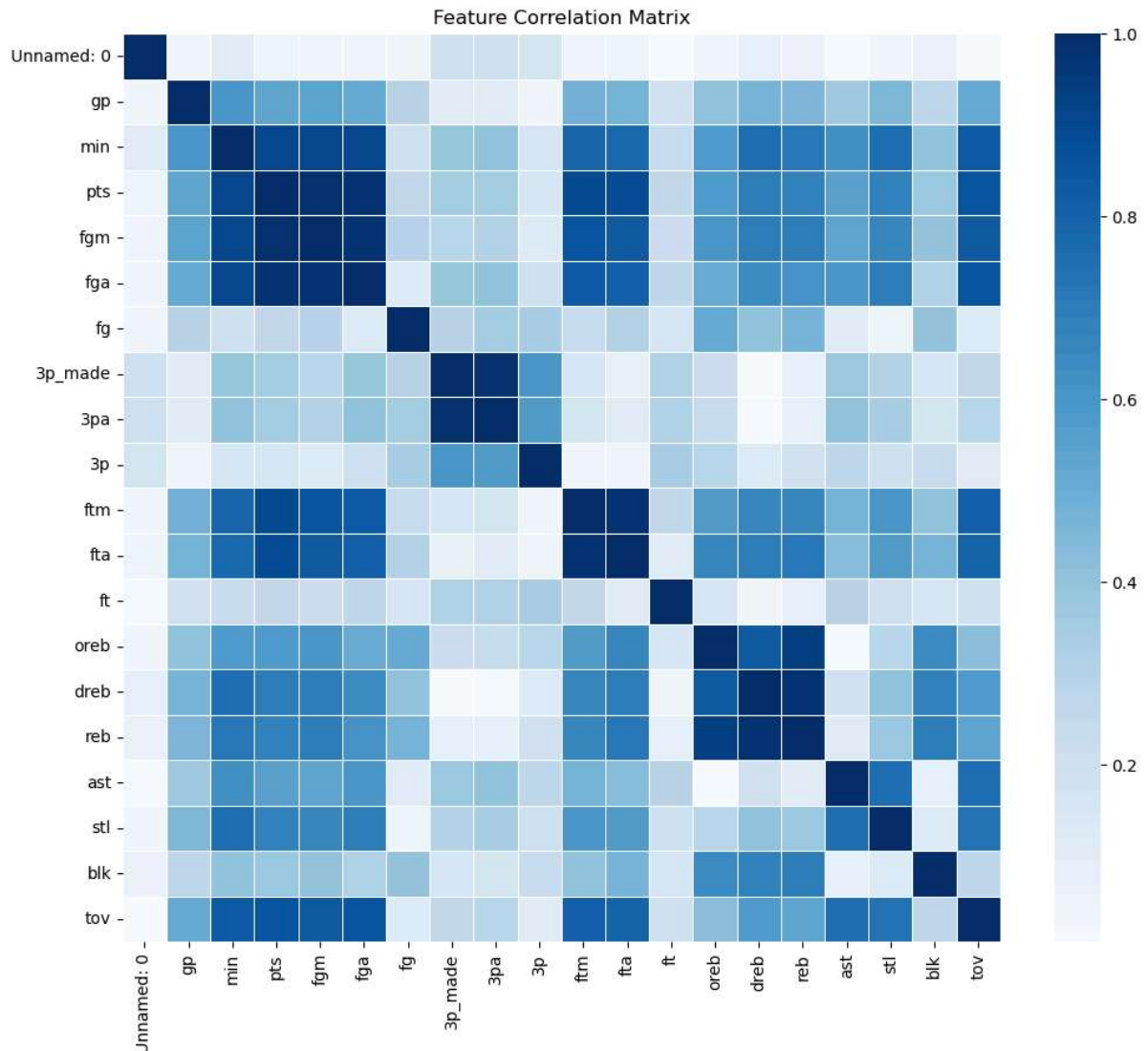
# Plot the heatmap
plt.figure(figsize=(12, 10))
sns.heatmap(corr_matrix, annot=False, cmap='Blues', linewidths=0.5)
```

```
plt.title("Feature Correlation Matrix")
plt.show()

# Select upper triangle of correlation matrix
upper_tri = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))

# Find features with correlation greater than 0.85
to_drop = [column for column in upper_tri.columns if any(upper_tri[column] > 0.85)]
print(f"Highly correlated features recommended for removal: {to_drop}")

# Drop highly correlated features if desired
# df_cleaned = df_cleaned.drop(columns=to_drop)
```



Highly correlated features recommended for removal: ['pts', 'fgm', 'fga', '3pa', 'ftm', 'fta', 'reb', 'tov']

Step 4: Feature Engineering

I Combine existing base variables into advanced composite metrics that give cleaner insight into player efficiency. I add domain-specific composite metrics to provide the predictive model with better signals.

1. **Points Per Minute (PPM):** Measures scoring efficiency relative to time on court.
2. **Player Efficiency Metric (Simple):** A rudimentary formula combining positive impacts (Points, Rebounds, Assists) and subtracting negative impacts (Turnovers).

```
In [4]: import numpy as np

# Prevent division by zero errors by converting 0 to NaN
if 'MIN' in df_cleaned.columns:
    df_cleaned['MIN'] = df_cleaned['MIN'].replace(0, np.nan)

# 1. Engineer Points Per Minute (PPM)
if 'PTS' in df_cleaned.columns and 'MIN' in df_cleaned.columns:
    df_cleaned['Points_Per_Minute'] = df_cleaned['PTS'] / df_cleaned['MIN']
else:
    print("Warning: 'PTS' or 'MIN' missing. 'Points_Per_Minute' not created.")

# 2. Engineer an Efficiency Metric
required_stats = ['PTS', 'REB', 'AST', 'TOV']
if all(stat in df_cleaned.columns for stat in required_stats):
    df_cleaned['Composite_Efficiency'] = (df_cleaned['PTS'] + df_cleaned['REB'] +
else:
    missing = [s for s in required_stats if s not in df_cleaned.columns]
    print(f"Warning: Missing columns {missing}. 'Composite_Efficiency' not created")

# Safely display only the columns that successfully exist in the DataFrame
display_columns = [col for col in ['Points_Per_Minute', 'Composite_Efficiency'] if

if display_columns:
    print(df_cleaned[display_columns].head())
else:
    print("Error: Neither engineered metric could be created.")
```

Warning: 'PTS' or 'MIN' missing. 'Points_Per_Minute' not created.

Warning: Missing columns ['PTS', 'REB', 'AST', 'TOV']. 'Composite_Efficiency' not created.

Error: Neither engineered metric could be created.

Step 5: Data Cleaning & Handling Nulls

Locate any gaps in the performance statistics and resolve them safely for machine learning algorithms. Machine learning models generally cannot handle missing values. I inspect the performance columns for null elements and impute them using the column median to avoid skewing the dataset with outliers.

```
In [5]: # Check for null values across the dataframe
null_counts = df_cleaned.isnull().sum()
print("Null values per column:")
print(null_counts[null_counts > 0])

# Impute performance statistics with their median value
performance_cols = df_cleaned.select_dtypes(include=[np.number]).columns.drop('tar
```

```

for col in performance_cols:
    if df_cleaned[col].isnull().sum() > 0:
        median_value = df_cleaned[col].median()
        df_cleaned[col] = df_cleaned[col].fillna(median_value)

print("\nData verification: Total remaining nulls =", df_cleaned.isnull().sum().sum())

```

Null values per column:
Series([], dtype: int64)

Data verification: Total remaining nulls = 0

Step 6: Final Selection & Summary Export

Prepare the target inputs and output configurations for extraction. The dataset has been successfully processed, cleaned, and enhanced with engineered features. It is now saved and ready to be fed into predictive modeling algorithms.

```

In [6]: # Separate into X (features) and y (target) to verify alignment
X = df_cleaned.drop(columns=['target_5yrs'])
y = df_cleaned['target_5yrs']

print(f"Final Feature Matrix Shape (X): {X.shape}")
print(f"Final Target Array Shape (y): {y.shape}")

# Optional: Save your engineered dataset to a new CSV file
# df_cleaned.to_csv('nba_players_engineered.csv', index=False)

```

Final Feature Matrix Shape (X): (1340, 21)
Final Target Array Shape (y): (1340,)